Python Fundamentals (Part1)

Concepts : Output/ Input, Variables, Data Types & Operators

Output

The first & the easiest thing that we can probably do in Python is printing (or outputting) something on our screen. And we can perform this simple task using a special function, called the `print()` function.
What exactly is a function? We'll learn about it in the next chapter but for now we can imagine it to be a specialized helper that performs a specific task. So, `print()` is our little helper that helps us display something on the screen(the console), anything. We can use it to show information, results of calculations or even format our text.

Syntax:

```
print("hello world")
# output will be 'Hello World'
```

Anything quotes('') or double-quotes("") is called a string & gets printed as it is.

We can also print numerical values:

```
print(3.14)      # output: 3.14
print(2 + 3)     # output: 5
```

Each print statement displays output on different lines but we can also display multiple values on same line.

```
print("hello world", "from PRIME")
```

Python Character Set

Character set is a collection of characters, numbers, symbols that our language recognizes & we can use. Following the common characters we'll be frequently using in Python:

1. English letters - `'A'` to `'z'` and `'a'` to `'z'`
2. Digits - `0` to `9`
3. Special symbols - `+`, `-`, `*`, `/`, `%`, etc.
4. Whitespaces & Escape characters - blank space ( ), tab space (`\t`), newline (`\n`) etc.
5. all ASCII & Unicode characters as part of data & literals

Variables

A **variable** is like a **container** that stores **data** or **values** in our program. We can think of it as a labeled box that we can put something in, change it, or use it later.
There variables in Python:
1. Have names called identifiers.
2. can store data of any type (numbers, text etc.)

Some **examples**:

```
name = "Shradha"
age = 30
PI = 3.14
word = "arificial intelligence"

print ("value of PI =", PI)
```

Variable Naming(identifier) Rules

1. Name must start with a **letter** (a–z, A–Z) or **underscore** `_` .
2. Can contain **letters, numbers, or underscores** after the first character.
3. Cannot use **Python keywords** (like `if` , `for` , `class`) as variable names.

Let's look at some invalid variable names:

```
2name = "Bob"    # Starts with a number
class = 10       # 'class' is a keyword
```

Important Note

1. Python is **dynamically typed**. So when creating variables we don't need to declare type explicitly.
2. Python is a **case-sensitive** i.e. variable `age` & `Age` are different.
3. **Indentation** in Python is important & the general standard is 4 spaces per indentation level.
4. **Keywords** are reserved words in Python & their meaning to the language is pre-defined.
   Examples - `if` , `for` , `class` , `while` , `else` , `in` , `def` etc.

Data Types

A **data type** defines the **kind of value a variable can hold**. Python has several built-in data types.
Let's have a look at the simplest ones to start with:

1. `int` - integer values (whole numbers)
2. `float` - floating-point values (decimal numbers)
3. `str` - strings (sequence of characters like words & sentences)
4. `bool` - boolean values (`True` / `False`)
5. `None` - integer values (whole numbers)

We also have a lot of other data types like `complex` , `list` , `tuple` , `set` etc. that we'll cover in coming chapters. To check the variable type we can use the `type()` function.

```
x = 10
print(type(x))          # <class 'int'>

PI = 3.14
print(type(PI))         # <class 'float'>

name = "shradha"
print(type(name))       # <class 'str'

isTeacher = True
print(type(isAdult))    # <class 'bool'>

empty_var = None
print(type(empty_var    # <class 'NoneType'>
```

Type Conversion & Type Casting

Type conversion is when we convert(cast) variables from one type to another. It can happen in 2 ways:

1. Type conversion - Implicit, done automatically by Python

```
a = 5
b = 3.0
print(a + b) # Python converts ans in float by default
```

2. **Type Casting** - Explicitly, done by the programmer

```
x = 10          # int
y = float(x)    # convert to float
z = str(x)      # convert to string
```

`int()` , `float()` , `str()` , `bool()` , `list()` , `tuple()` are common type conversion functions.

Operators

An **operator** is a symbol that tells Python to perform a **specific computation** on one or more values (called operands).
Example:

```
print(1 + 3) # '+' is an Operator & (1, 3) are operands
```

There are several types of operators in Python. Let's start by understanding some of the most common ones:

1. **Arithmetic Operators** - used to perform math operations.

   Operators - `+` , `-` , `*` , `/` , `%` , `**`

```
a = 5
b = 10

print(a + b)     # Addition
print(a - b)     # Subtraction
print(a * b)     # Multiplication
print(a / b)     # Division
print(a % b      # Modulo - remainder
print(a ** b)    # Power
```

2. Relational Operators - used to compare values.

Operators - `==` , `!=` , `>` , `>=` , `<` , `<=`

```
a = 10
b = 20

# Equal to
print("a == b:", a == b) # False

# Not equal to
print("a != b:", a != b) # True

# Greater than
print("a > b:", a > b) # False

# Less than
print("a < b:", a < b) # True

# Greater than or equal to
print("a >= b:", a >= b) # False

# Less than or equal to
print ("a <= b:", a <= b) # True
```

3. Assignment Operators - used to assign values to variables.

Operators - `=` , `+=` , `-=` , `*=` , `/=` , `%=` , `**`

```
# Simple assignment
x = 10
print("x =", x) # 10

x += 5 # equivalent to x = x + 5
print("x += 5 ->", x) # 15

x -= 3 # equivalent to x = x - 3
print("x -= 3 ->", x) # 12

x *= 2 # equivalent to x = x * 2
print("x *= 2 ->", x) # 24

x /= 4 # equivalent to x = x / 4
print("x /= 4 ->", x) # 6.0

x %= 4 # equivalent to x = x % 4
print("x %= 4 ->", x) # 2.0

x **= 3 # equivalent to x = x ** 3
print("x **= 3 ->", x) # 8.0
```

4. Logical Operators - used to combine boolean values.

Operators - `and` , `or` , `not`

val1	val2	and
T	T	T
T	F	F
F	T	F
F	F	F

val1	val2	or
T	T	T
T	F	T
F	T	T
F	F	F

val1	not
T	F
F	T

```python
x = 10
y = 20
z = 5

# AND
print(x > z and y > x) # True

# OR
print(x > y or y > z) # True

# NOT
print(not (x > y)) # True
```

We'll cover more operator types like membership operators in future chapters.

Operator Precendence

Operators have a **priority** i.e there is a specific order in which operations are to be performed in case we have multiple operators in the same expression. The order is as follows:

- `()` - Parentheses (highest priority)
- `**` - Exponent
- `+x, -x, ~x` - Unary operators
- `/ // %` - Multiplication, division, floor, modulus
- `+ -` - Addition, subtraction
- `<< >>` - Bitwise shifts
- `&` - Bitwise AND
- `^` - Bitwise XOR
- `|` - Bitwise OR
- Comparison operators - `<, <=, >, >=, !=, ==`
- `not`
- `and`
- `or`
- Assignment operators - `=, +=, -= ...`

Note - Unary operators are operators that **works on a single operand** (a single value or variable) to perform an operation. These are like `not`, `+` (unary plus), `-` (unary minus) etc.
`+x` is used to indicate positive value & `-x` to indicate negative value.

```
x = 5
y = -x # unary minus
print(x) # 5
print(y) # -5
```

We don't generally use +x as numbers are positive by default.

Input

We use the `input()` function to take input from user while the program is running. Whatever the user types in input prompt is returned as a string.

Syntax:

```
name = input("enter name: ")
print(name)
print(type(name)) # type is string
```

We can use type conversion functions to convert the type of input.

```
# program to add 2 numbers entered by the user
a = int(input("enter 1st num: "))
b = int(input "enter 2nd num: "))

print(a + b)
```

Let's summarize all the concepts we learnt in this chapter into a simple practice problem.
Write a program that takes in 2 numbers (`a` & `b`) and print the average.

```
a = int(input("enter 1st num: ")) # 5
b = int(input("enter 2nd num: ")) # 10

avg = (a + b) / 2
print("average = ", avg) # 7.5
```

Keep Learning & Keep Exploring!

Python Fundamentals (Part2)

Concepts : Conditional Statements, Loops & Functions

Conditional Statements

Conditional statements let our program **decide what to do based on conditions** (i.e. true or false expressions).

There are 3 main conditional statements in Python:

1. `if` - used to test a condition. If it's **True**, the indented block runs.
2. `else` - else block runs if all above conditions are false.
3. `elif` - ("else if") used when we have **multiple conditions** to check.

Let's practice conditional statements with the help of following examples.

```
# Example 1
age = int(input("enter age: "))

if age >= 18:            # if true/false, entire block of code is executed
    print("you can vote")
    print("you can drive")

else:
    print("you can't vote")
    print("you can't drive")
```

```
# Example 2 - Traffic Lights
color = input("enter color: ")

if color == "red":
    print("Stop")
elif color == "yellow":
    print("Look")
elif color == "green":
    print("Go")
else:
    print("wrong color")
```

Note - We can have standalone & multiple `if` statements but `elif` & `else` can't be used without an `if` statement preceding them.

Apart from simple conditions we can also check for multiple conditions using Logical operators like `and` , `or` and `not` in expressions.

```
# Example 3 - Logical operators in expressions
age = int(input("enter age: "))

if age < 13:
    print("child")
elif (age >= 13 and age < 18):
    print("teenager")
else:
    print("adult")
```

Nesting in Conditionals

Nesting means **placing one block of code inside another**. In the context of conditional statements, it means putting one `if` (or `if-elif-else`) inside another. This helps when you need to make a decision based on another decision.

```
# Login System

username = input("enter username: ")
password = input("enter password: ")

if (username == "admin" and password == "pass"):
   print("log in successful!")
else:
 if username != "admin":          # NESTING
        print("wrong user name, try again.")
    else:
        print("wrong password, try again.")
```

A login system that checks for correct/ incorrect credentials

Ternary Statements

Ternary statements (or conditional expressions) are just another compact way to writing our `if-else` statements in one-line.

Ternary statement **Syntax**:

```
value_if_true if condition else value_if_false
```

One line decided between 2 values based on condition.

Let's see an example:

```
age = 18
status = "Adult" if age >= 18 else "Not Adult"
print(status)
```

Note - Everything that we write with ternary statements can be done with normal if-else statements. Ternary statements are generally only used for simple conditions, not recommended for nested or complex conditions.

Match Case Statements

In Python, the `match` / `case` statements are an alternative to long chain of `If..elif..else` statements. They were introduced in Python 3.10 & we generally don't see them much in production code.

Match case **Syntax**:

```
match variable:
   case pattern1:
        # code to run if pattern1 matches
   case pattern2:
       # code to run if pattern2 matches
   case _:
       # default case (like 'else')
```

1. `match` - what you're comparing
2. `case` - the value or pattern to match against
3. `_` - wildcard (matches anything, like "default")

Let's see an example:

```
color = input("enter color: ")

match color:
    case "red":
       print("Stop")
    case "yellow":
       print("Look")
    case "green":
       print("Go")
    case _:
      print("wrong color")
```

Simple value match for traffic lights

Practice Problems (Set 1)

1. For a given number `n` , print if it's a multiple of 5 or not.
2. For a given number `n` , print if it's Odd or Even.

Loops

A **loop** lets us **execute a block of code multiple times:** either a set number of times or until a condition is met.

Python has 2 main types of loops:

1. `while` loop - repeat while a condition is `True`
2. `for` loop - iterate over a sequence (like a list, string, or range)

The while Loops

Syntax of a `while` loop:

```
while condition:
    # code block
```

Whatever statements we write in the code block are all executed one after the other

Let's look at some examples:

```
# Example 1 - print 1 to 5
i = 1
while I <= 5:
    print(i)
    i += 1

# Example 2 - print 5 to 1
i = 5
while i > 0:
    print(i)
    i -= 1
```

If the loop expression is such that it always computes `True`, then such a loop never stops & is called an **infinite loop**. We should try to never unintentionally create such a loop.

```python
while True: # DO NOT RUN this code
    print("Prime")
```

Example of an Infinite Loop

Loop control statements

Python gives us some tools to **control** how loops behave & a classical example of them are 2 statement - `break` & `continue`.

The `break` keyword

It stops the loop immediately.

```python
# Break for multiple of 6
i = 1

while i <= 10:
    if(i % 6 == 0):
        break
    print(i)
    i += 1

# output: 1, 2, 3, 4, 5
```

The `continue` keyword

It skips the rest of the current iteration and moves to the next one.

```python
# Skip multiples of 3
i = 0

while(i < 10):
    i += 1
    if(i % 3 == 0):
        continue;
    print(i)

# output: 1, 2, 4, 5, 7, 8, 10
```

The for Loops

In Python, a `for` loop is used to iterate (loop) over a sequence, such as a list, tuple, string, or range, and execute a block of code for each item in that sequence.

Syntax of a `for` loop:

```
for variable in sequence:
    # code to run for each item
```

Let's look at an example:

```
for i in range(5):
    print(i)

# output: 0, 1, 2, 3, 4
```

The `in` keyword that we used is a **membership operator** in Python, used to loop over a sequence or to check the presence of an item in a sequence, like a string.

```
word = "Prime"

# Example 1 - Looping over a string
for ch in word:
    print(ch)

# Example 2 - Check if char 'i' exists in word
if 'i' in word:
    print("letter exists")

# Example 3 - Count number of i's in the word
word = "artificial intelligence"
count = 0

for ch in word:
    if ch == 'i':
        count += 1

print(f"i occurs {count} times.")
```

Just like conditionals, we can also have **nested loops** i.e. loop inside a

```
for i in range(1, 3):
    for j in range(1, 3):
    print(f"({i}, {j})")

# output: (1, 2) (1, 2) (2, 1) (2, 2)
```

Nested Loop Example

The `range()` Function

The `range()` function in Python is used to generate a **sequence of numbers**, typically for use in loops. It doesn't actually create a list in memory (unless we convert it); instead, it creates an iterator that produces numbers one by one, which makes it very efficient.

The function has 3 parameters:

1. start (default=0) - the number to start from
2. stop - the number to stop before (not included)
3. step (default=1) - how much to increase by each time

```
# single argument - start
for i in range(5):
    print(i)

# output: 0, 1, 2, 3, 4

# 2 arguments - start, stop
for i in range(1, 6):
    print(i)

# output: 1, 2, 3, 4 , 5

# 3 arguments - start, stop, step
for i in range(1, 10, 2):
    print(i)

# output: 1, 3, 5, 7, 9
```

Practice Problems (Set 2)

1. Print multiplication table for any number `n` . [using `while`]
2. Print odd numbers from 1 to 10, using continue. [using `while`]
3. Count vowels in a word. [using `for`]
4. Sum of first `n` natural numbers. [using `for`]

Functions

Functions in Python are **blocks of reusable code** that perform a specific task. They make our code organized, readable, and easier to maintain.

Function Definition

We use the `def` keyword to define our function:

```
# Function Definition
def hello():
    print("hello from Prime")
```

and to call the function i.e. to run it we write their name with parentheses:

```
hello() # Function Call
```

Function call actually invokes the function & then all the block of code written in the functions definition is executed.

Functions with Parameters

We can also pass data to a function using **parameters** (inside the parentheses). Values actually passed when calling the function are called **arguments**.

```
# Fnx to compute sum of 2 nums

def sum(a, b):          # a & b are parameters
    print (a + b)

print(sum(5, 10))       # 5 & 10 are arguments
```

wherever fnx → it means function

Functions can **return** a result using the `return` keyword.

```
# Fnx to computer average of 3 nums

def avg(a, b, c):
    return (a + b + c) / 3
print(avg(1, 2, 3))
```

Default Parameters

We can assign **default values** to function parameters so that if the caller doesn't provide that argument, Python uses the default values. And these default parameters must come last. We cannot have a non-default argument after a default argument.

```
# sum() fnx with default param 1
def sum(a, b = 1): # default val of b is 1
    return a + b

print(sum(5)) # output: 6
```

Built-in Functions

All the functions that we have studied till now are **user-defined functions**, but in Python we also have a lot of built-in functions too. The definition (logic) of **built-in functions** is already written in Python & we just have to use them.
Some built-in functions that we have already used till now:

1. `print()`
2. `input()`
3. `type()`
4. `range()`

Lambda Functions

Lambda functions are short, one-liner functions that are used to perform simple tasks. These are created using the `lambda` keyword. These functions are anonymous & do not have a name like regular functions defined with `def` .
Lambda function **Syntax**:

```
lambda arguments: expression
# arguments -> 1 or more parameters
# express ion -> single expression whose result is automatically returned
```

Let's see an example of a lambda function that computes square of a number:

```
# fnx to compute x^2
square = lambda x: x * x
print(square(5))
```

Practice Problems (Set 3)

1. Create a function to compute factorial of a number `n` .

2. Create a function to return largest of 3 numbers - `a` , `b` & `c` .

Solutions (Set 1 - Conditionals)

1. Multiple of 5 or not

```
num = int(input("enter number: "))

if num % 5 == 0:
    print("multiple of 5")
else:
    print("NOT a multiple of 5")
```

2. Odd or Even

```
num = int(input("enter number: "))

if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Solutions (Set 2 - Loops)

1. Multiplication table of `n`

```
n = int(input("enter n: "))
i = 1

while i <= 10:
      print(i * n)
      i += 1
```

2. Odd nums from 1 to 10, using continue

```
i = 0

while(i < 10):
      i += 1
      if(i % 2 == 0):
            continue;
      print(i)

# output: 1, 3, 5, 7, 9
```

3. Count vowels

```
for ch in word:
    if (ch   == 'a' or ch == 'a' or ch == 'a' or ch == 'a' or ch == 'a'):
        count += 1

print(f"vowel count = {count}")
```

4. Sum of first n Natural numbers

```
n = int(input("enter n: "))
sum = 0
for i in range(1, n+1):
    sum += i

print("sum = ", sum)
```

Solutions (Set 3 - Functions)

1. Factorial of N

```python
# Factorial of N
n = int(input("enter n: "))

fact = 1
for i in range (1, n+1):
fact *= i

print("factorial = ", fact)
```

2. Largest of 3 numbers

```python
def get_largest (a, b, c):
    if (a > b and a > c):
        return a
    elif b > c:
        return b
    else:
        return c
print (get_largest (3, 10, 5))
```

Keep Learning & Keep Exploring!

Python Fundamentals (Part3)

Concepts : String, List, Tuple, Dictionary & Set

In this chapter we are going to dive deep into some of the not-so-simple data types of Python.

Strings

What is a String?

A **string** in Python is a sequence of characters enclosed in quotes:

```
str1 = "hello world"
str2 = 'Prime'
```

Strings are **immutable**, meaning once created, their contents can't be changed directly.

`len()` Function

We can use the built-in `len()` function to print the length of a string:

```
word = 'Prime'
print(len(word))        # 5
```

Concatenation

Just like we add numbers, we can concatenate (i.e. join) strings using the `+` operator.

```
str1 = "Apna"
str2 = "College"

word = str1 + " " + str2        # concatenation
print(word)
```

Loop on Strings

```
s = "Python"
for ch in s:     # ch will store individual chars - 'P', 'y', 't' & so on
    print(ch)
```

Indexing in Strings

Index in a sequence (like strings) represents the position value of individual elements i.e. characters in case of a string. So Indexing lets us access individual characters in a string.

Python follows **Zero-based indexing** i.e. first character is at index `0`.

```
s = "Python"
print(s[0])  # 'P'
print(s[3])  # 'h'
print(s[-1]) # 'n' (negative index: last character)
```

Slicing in Strings

Slicing is a powerful feature of Python that lets us access multiple elements at once. We can do slicing in strings & even on other sequences like lists & tuples.
The general syntax for slicing a string is:

```
string[start : ~~end~~ stop : step]
```

Where:

- **start** - index where the slice starts (inclusive). Defaults to `0` if omitted.
- **stop** - index where the slice ends (exclusive). Defaults to the end of the string if omitted.
- **step** - how many indices to move forward each time. Defaults to `1`.

Example:

```
s = "Python"
print(s[0:2])   # 'Py'
print(s[2:])    # 'thon'
print(s[:3])    # 'Py t'
print(s[::2])   # 'Pto' (every second char)
print(s[::-1])  # 'nohtyP' (reversed string)
```

String Formatting

String formatting is the process of creating **dynamic strings** by inserting values from variables or expressions into a predefined string template. This allows for the construction of flexible and informative output based on changing data.
We have multiple ways to format a string, the most modern 2 are:

1. using the `format()` function
2. using `f-strings`

1. Using `.format()`

In this way we use `{}` as placeholders & pass placeholder replacement values in the format function.

```
name = "Rahul"
age = 25

text = "My name is {} and I am {} years old".format(name, age)
print(text)
```

We can also use positional & named placeholders:

```
"Coordinates: {1}, {0}".format("x", "y") # 'Coordinates: y, x'

"Name: {n}, Age: {a}".format(n="Bob", a=30)
```

2. **Using f-strings** (Python 3.6+)
F-strings are concise, readable & will be our preferred way going forward.
In f-strings we prefix the string with `f` and put our variable or expressions inside `{}` .

```
name = "Rahul"
age = 25
text = f"My name is {name} and I am {age} years old"
print(text)
```

You can put any valid Python expression inside `{}` :

```
a = 5
b = 10
print(f"sum of {a} & {b} = {a + b}")
print(f"avg of {a} & {b} = {(a + b) / 2}")
```

Lists

What is a List?

- A **list** is a **collection of items** in Python.
- Items in a list are **ordered, changeable (mutable), and can contain duplicates**.
- Lists can hold any data type: numbers, strings, other lists, etc.
- Lists are written using square brackets `[]`

Example:

```
my_list = [1, 2, 3, 4, 5]
print(my_list)
print(type(my_list))     # <class 'list'>

my_list2 = [10, "Hello", 3.14, True, 10]     # heterogenous list
print(my_list2)
```

List Characteristics

1. **Ordered** – Items have a defined order and can be accessed by index.
2. **Mutable** – Items can be changed after the list is created.
3. **Allows duplicates** – Same value can appear more than once.
4. **Heterogeneous** – Can contain different data types.

List Indexing

Index in list is the position value of an item. Index starts from `0` .
We can use index to access elements, modify the list or even slice it.

Access Elements

```
my_list = ["apple", "banana", "cherry"]
print(my_list[0])  # apple
print(my_list[1])  # banana
print(my_list[-1]) # cherry (last element)
```

Modify Elements

```
my_list = [1, 2, 3, 4]
my_list[0] = 10
print(my_list) # [10, 2, 3, 4]
```

Slicing - Slicing in lists is same as slicing in strings.
The general syntax for slicing a list is:

```
list[start:stop:step]
```
(stop → end)

- **start** - inclusive
- **end** - exclusive
- **step** - optional (default = 1)

```python
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Simple Slice
print(numbers[2:5])  # Output: [2, 3, 4]

print(numbers[:4])   # Output: [0, 1, 2, 3] (from start to index 3)
print(numbers[5:])   # Output: [5, 6, 7, 8, 9] (from index 5 to end)
print(numbers[:])    # Output: [0,1,2,3,4,5,6,7,8,9] (copy of the whole list)

# using STEP
print(numbers[::2])  # Output: [0, 2, 4, 6, 8] (every2nd element)
print(numbers[1::3]) # Output: [1, 4, 7] (start at 1, every 3rd element)

# NEGATIVE slice
print(numbers[-5:-2]) # Output: [5, 6, 7] (negative indexing from end)
```

List Methods

We have a lot of useful functions that are associated with lists. Let's have a look at some of them:

1. `len()` - returns the number of elements in a list i.e. length of the list.
2. `append(element)` - adds an elements to the end.
3. `insert(element, idx)` - adds at a specific position.
4. `sort()` - sorts in ascending/alphabetical order.
5. `reverse()` - flips the order of elements.

```python
nums = [5, 2, 9]
print(len(nums)) # 3

nums.append(7)
print(nums) # [5, 2, 9, 7]

nums.insert(1, 4)
print(nums) # [5, 4, 2, 9, 7]

nums.sort()
print(nums) # [2, 4, 5, 7, 9]

nums.reverse()
print(nums) # [9, 7, 5, 4, 2]
```

There are many more methods associated with lists. It doesn't make a lot of sense to cover all of them theoretically here, so we'll be covering them whenever we use them in coming chapters.

Loops on Lists

We use the classical `for` loop to traverse the entire list element-wise.

The most common way is using a loop in lists is to print elements.

```
numbers = [10, 20, 30, 40, 50]

for num in numbers:
    print(num)
```

Linear Search

A linear search checks each element one by one to find a target (x).

```
numbers = [5, 12, 7, 3, 18, 9]
x = 18
idx = 0

for num in numbers:
    if num == target:
        print(f"{x} found at index={idx}")
        break
idx += 1
```

idx += 1

In the above code, we are assuming that `x` always exists in the list.

Tuples

What is a Tuple?

A **tuple** in Python is an ordered, immutable collection of items.
Example:

```
tup = (10, 20, 30)

print(tup)
print(type(tup)) # < class 'tuple'>

empty_tu ple = ()   # empty_tuple

single_element_tuple = (42,)
```

Tuple Characteristics

1. Ordered
2. **Immutable** – Items cannot be changed after the tuple is created.

3. Allows duplicates
4. Heterogeneous

> Tuples are very similar to lists. Some of the differences are that tuples are immutable & because of this immutability they are also faster.

Indexing & Slicing (Same as Lists)

```
t = (10, 20, 30, 40)

print(t[0])   # 10
print(t[-1])  # 40
print(t[1:3]) # (20, 30)
```

Loops on Tuples (Same as Lists)

```
t = (10, 20, 30, 40)

for val in t:
    print(val)
```

Using loops to calculate **sum** of all elements in the tuple:

```
t = (5, 15, 25)

sum = 0
for val in t:
    sum val

print("Sum:", sum)
```

Tuple Methods

Let's have a look at some of tuple methods:

1. `index(val)` - returns index of first occurrence for any val
2. `count(val)` - returns total count of occurrence for any val

```
t = (1, 2, 2, 3, 5)

print(t.index(2)) # 1
print(t.count(2)) # 3
```

Dictionaries

What is a Dictionary?

A dictionary is an unordered, mutable collection of **key-value pairs**.
Example:

```
my_dict = {
    "name": "Shradha",
    "age": 30,
    "city": "Delhi"
}
```

Dictionary Characteristics

1. Dictionary Keys must be **unique**
2. Dictionary Keys must be **immutable** (e.g., strings, numbers, tuples)
3. Dictionary is mutable.
4. Dictionary values can be anything (lists, other dictionaries, etc.)
5. Unordered (although in modern Python, dictionaries preserve insertion order)

Accessing values (using key & `[]`)

```
student = {"name": "Bob", "age": 20}
print(student["name"])  # Bob
```

Dictionary Methods

Let's have a look at some of the important dictionary methods:

1. `keys()` - returns all keys
2. `values()` - returns all values
3. `items()` - returns key–value pairs as tuples
4. `get(key)` - a safer way to access value of a particular key. Instead of throwing an error it returns `None` if key doesn't exist
5. `update(new_item)` - adds a new item to the dictionary

```
d = {
    "name": "Shradha,
    "subjects"
    "cgpa": 9.5
}

print(d.keys()) # dict_keys
print(d.values()) # dict_values
print(d.items()) # dict_items

print(d.get("cgpa2")) # return None as no such key as "cgpa2"

new_item = {"city": "Delhi"}
print(d.update(new_item))
print(d)
```

Loops on Dictionary

We can loop through using key-pair values:

```
d = {
    "name": "Shradha,
    "cgp a": 9.5
}

for key, value in d.items():
    print(key, value)
```

Sets

What is a Set?

A set is an unordered collection of **unique elements**.
Example:

```
my_set = {1, 2, 2, 2, 3}

print(my_set)           # {1, 2, 3}
print(type(my_set))
print(len(my_set))      # 3


empty_set = set()
```

Set Characteristics

1. Sets can only have **unique** elements.
2. They are **unordered** - no indexing or slicing.
3. They are **mutable** - we can add or remove elements.
4. Set elements must be **immutable** (like strings, numbers, tuples).

Sets are often used when we need uniqueness or mathematical set operations.

Set Methods

Let's have a look at some of the important set methods:

1. `add(val)` - adds an element to set
2. `remove(val)` - removes an element (raises error if not found)
3. `clear()` - removes all elements
4. `pop()` - removes and returns a random element (since sets are unordered)
5. `s1.union(s2)` - returns new union (union is collection of all unique values in both sets)
6. `s1.ntersection(s2)` - returns new union (intersection is collection of all common & unique values in both sets)

```
s = {10, 20, 30}

s.add(40)       # {10, 20, 30, 40}
print(s)

s.remove(10)    # {20, 30, 40}
print(s)

print(s.pop())  # can be any value

s.clear()
print(s)        # set() - empty set

# Union & Intersection
A = {1, 2, 3}
B = {3, 4, 5}

print(A.union(B))         # {1, 2, 3, 4, 5}
print(A.intersection(B))  # {3}
```

> *Keep learning & Keep exploring!*

Python Fundamentals (Part4)

Concepts : Object Oriented Programming in Python

Object Oriented Programming

Let's suppose we need to build a software to store all the information related to students studying in our college.

These students could have a lot of **properties** (like name, parent's name, address, graduation year, cgpa etc.) associated with them.

They could also have a lot of **behaviours** (like fee payment, scholarship calculation, checking minimum attendance criteria etc.) associated with them.

Without existing Python data types, we can choose to store this information in lists or dictionaries but that would not be efficient. As we have a lot of properties, we'd have to create too many lists and it'll be hard to keep track of them. If we use dictionaries, we'd have to re-define the same keys again & again & we won't logically be able to associate behaviours with that data.

So to simplify our work of creating such a software we have **Object Oriented Programming** - which gives us the concept of **classes** & **objects**.

What is a Object Oriented Programming (OOP)?

Object-oriented programming (OOP) in Python is a programming paradigm centered around organizing code into **objects:** bundles of data (attributes) and behaviour (methods). It helps us structure programs in a way that is modular, reusable, and easier to maintain.

Note - It's not compulsory to always use OOP concepts but they definitely help in a lot of cases.

OOP models software as a collection of interacting **objects**, similar to how we think about real-world entities. Each object represents something meaningful in our program - such as a *user*, a *car*, a *bank account*, or an *enemy* in a video game.

In Python, OOP is based around **classes**, which are blueprints for creating objects.

What are Classes & Objects?

Class

A class is a blueprint or template for creating objects. We can think of a class as a recipe: it describes what an object will look like (its attributes) and what it can do (its methods), but it is not the object itself.

```python
class Car:
    brand = "Toyota"
```

Object

An **object** (or **instance**) is a realization of a class, it is the actual thing created based on the class blueprint. Now based on the template we can create as many objects as we want.

```python
car1 = Car()
car2 = Car()

print(car1.brand)      # Toyota
print(car2.brand)      # Toyota
```

We use the "." (Dot Operator) to access properties & methods of objects.

Class vs. Object

Class	Object
Class is a blueprint/ template.	An object is a concrete instance of a class.
Does not exist in memory until instantiated.	Contains actual data & occupies memory.
One class can create any number of objects.	Each object is independent.

Attributes & Methods

Attributes are variables and **Methods** are functions defined inside a class.
(We'll cover both in detail)

Constructor in OOP

In Python, a **constructor** is a special method used to initialize newly created objects. It is not responsible for creating the object, instead the constructor sets up the object with initial values.

We use the `__init__(self, ...)` method to define our constructor. Whenever we create an object of a class, Python automatically calls the `__init__()` method.

```python
class Student:
    def __init__(self):
        print("constructor was called")

stu1 = Student()         # "constructor was called"
```

`self` is a special parameter which refers to the **instance of the class** that is calling the method. We don't need to pass it explicitly.

We can also use constructor to initialize values for objects:

```
class Student:
    def __init__(self, name):
        self.name = name

stu1 = Student("Rahul")
stu2 = Student("Harshita")

print(stu1.name, stu2.name)   # Rahul Harshita
```

Types of Constructors

1. **Default Constructors** - A constructor with **no parameters** except `self`.
2. **Parameterized Constructors** - Takes parameters to initialize values uniquely for each object.

> Note - Python doesn't support constructor overloading directly (like Java/C++) i.e. having multiple constructors in the same class. Whichever is written last is executed.

Attributes in OOP

Attributes are **variables** that belong to a class or an object. They store data/state of the object.

Types of Attributes

1. **Class Attributes**

- Belong to the **class itself**, shared by *all* objects.
- Defined **outside** any method in the class.

```
class Student:
    college = "ABC college"          # class attribute

stu1 = Student ()

print(stu1.college)
print(Student.college) # class attribute can also be accessed with class name
```

2. Instance Attributes

- Belong individually to **each object**.
- Defined inside the `__init__` method using `self` .
- Each object gets its own copy.

```
class Student:
    def _init_(self, name, gpa): # instance attributes
        self.name = name
        self.gpa = gpa

stu1 = Student("Rahul", 8.7)
print(stu1. name, stu1.gpa)
```

Methods in OOP

Methods are **functions defined inside a class**, representing the **behaviour** or **actions** of an object.

Types of Methods

1. Instance Methods

- Take `self` as the first argument.
- Can access both **instance attributes** and **class attributes**.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self. marks = marks

    def display(self):          # Instance method
        print(f"Name: {self.name}, Marks: {self.marks}")
```

2. Class Methods

- Use `@classmethod` decorator.
- Take `cls` (class) as first argument.
- Used to work with **class-level data**.

```
class Student:
    school_name = "ABC School"

    @classmethod
    def change_school(cls, new_name):
        cls.school_name = new_name
```

3. Static Methods

- Use `@staticmethod` decorator.
- Do **not** take `self` or `cls` .
- Behave like normal functions but belong to the class for logical grouping.

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b
```

OOP Pillars

Let's understand the 4 Key pillars of OOP - Encapsulation, Abstraction, Inheritance & Polymorphism.

Encapsulation

Encapsulation is the bundling of data (variables) and methods (functions) that operate on that data into a single unit (a class), along with controlling access to that data. This is done to protect the data from accidental or unauthorized modification.

To implement encapsulation, we use access modifiers. Python has 3 access levels:

1. Public members

- Accessible everywhere, written like normal variables.

```
class Student:
    def __init__(self, name):
        self.name = name  # public variable

s = Student("Rahul")
print(s.name)  # Allowed
```

2. Protected members

- Indicated by a **single underscore** `_` (suggest - "Don't access directly unless needed.")
- Still accessible from outside (not truly protected).
- Intended for internal use or inheritance.

```python
class Person:
    def __init__(self):
        self._age = 20  # protected variable

p = Person()
print(p._age)  # Technically allowed, but not recommended
```

3. Private members

- Indicated by a **double underscore** `__`
- Python does *name mangling*: the variable name becomes `_ClassName__variable`.
- Cannot be accessed directly from outside.

```python
class Bank:
    def __init__(self, balance):
        self.__balance = balance # private variable

b = Bank(5000)
print(b.__balance)  # ERROR: attribute not accessible
```

To access:

```python
print(b._Bank__balance) # Allowed (name-mangled form)
```

Python uses naming conventions with access modifiers, not strict enforcement (like Java/C++).

Getters & Setters

When we make a variable private, we use methods to read it (getters) or update it (setters).

```python
class Employee:
    def __init__(self, salary):
        self.__salary = salary       # private

def get_salary(self):                # getter
    return self.__salary

def set_salary(self, new_salary):    # setter
    self.__salary = new_salary

e = Employee(50000)
print(e.get_salary())
e.set_salary(60000)
```

Inheritance

Inheritance is where one class (child) acquires the properties and behaviors (variables + methods) of another class (parent).

The class whose properties are inherited - **Parent** / Base / Superclass

The class that inherits - **Child** / Derived / Subclass

```
class Employee:              # parent class
    start_time = "9AM"
    end_time = "5PM"

class Teacher(Employee):     # child class
    def __init__(self, subject):
        self.subject = subject

t1 = Teacher("Data Science")

print(t1.subject, t1.start_time, t1.end_time)
```

Inheritance enables:

- Code reuse
- Extensibility
- Cleaner, maintainable design
- Polymorphism

Types of Inheritance

1. Single Inheritance

A child inherits from one parent.

```
# Parent → Child

class Parent:
    def display(self):
        print("Parent class")

class Child(Parent):
    pass

c = Child()
c.display() # Output: Parent class
```

2. Multi-level Inheritance

A child inherits from a parent, and another class inherits from the child.

```
# Grandparent(Employee) → Parent(AdminStaff) → Child(Accountant)

class Employee:
    start_time = "9AM"
    end_time = "5PM"

class AdminStaff(Employee):
    def __init__(self, role):
        self.role = role

class Accountant(AdminStaff):
    def __init__(self, salary, role):
        super().__init__(role)
        self.salary = salary

acc1 = Accountant(50_000, "CA")

print(acc1.salary, acc1.role, acc1.start_time, acc1.end_time)
```

3. Multiple Inheritance

A child inherits from more than one parent class.

```
class Teacher:
    def __init__(self, salary):
        self.salary = salary

class Student():
    def __init__(self, gpa):
        self.gpa = gpa

class TA(Teacher, Student):
    def __init__(self, name, salary, gpa):
        super().__init__(salary)            # call parent constructor
        Student.__init__(self, gpa)         # call parent constructor
        self.name = name

ta = TA("Rahul", 50_000, 7.5)

print(ta.name, ta.salary, ta.gpa)
```

super() keyword - Used to call parent class's method from child class.

Abstraction

Abstraction is hiding unnecessary implementation details and showing only the essential features to the user.

Example - In real life when we drive a car & press the breaks, the car stops. But we don't need to know how the hydraulic systems work. To implement same idea in Python, we have abstraction.

We implement abstraction with abstract classes & abstract methods.

Abstract Class

An abstract class in Python is one which:

- Cannot be instantiated
- Can contain normal + abstract methods
- Usually acts as a blueprint for child classes

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass
```

Abstract Method

It is a method declared but not implemented (Children **must** override abstract methods).

```
@abstractmethod
def method_name(self):
```

Example of Abstraction

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound():
        pass

class Lion(Animal):
    def make_sound(self):
        print("Roar!")

class Cow(Animal):
    def make_sound(self):
        print("Moo!")

lion = Lion()
lion.make_sound()

cow = Cow()
cow.make_sound()
```

Polymorphism

Polymorphism is the ability of a single function, operator, or object to behave differently based on the context. (“poly” = many, “morph” = forms)

The idea is that:

- Same method name - works differently for different objects
- Same operator - behaves differently depending on operand types

Let’s look at an example:

```
print(1 + 2)     # adds 2 numbers
print("1" + "2") # concatenates 2 strings
```

Same ‘+’ operator being used for 2 different operations, called **Operator Overloading.**

Let’s look at two popular types of polymorphism:

1. **Function Overriding** (or Method Overriding)

- When a child class provides its own version of a method that already exists in the parent class (Both methods should have same name)
- Type of Runtime Polymorphism (dynamic binding)
- Child method **takes precedence** over parent method.

```
class Animal:
    def sound(self):
        print("Some generic sound")

class Dog(Animal):
    def sound(self):
        print("Bark")

a = Animal()
dog = Dog()

a.sound() # Some generic sound
dog.sound() # Bark
```

2. Duck Typing

- Works on the idea: *"If it looks like a duck and quacks like a duck, it must be a duck."*

```
class Dog:
    def speak(self):
        print("Bark")

class Cat:
    def speak(self):
        print("Meow")

class Robot:
    def speak(self):
        print("Beep Boop")

def make_it_speak(entity):
    entity.speak()  # doesn't care about type

d = Dog()
c = Cat()
r = Robot()

for e in [d, c, r]:
make_it_speak(e)
```

> *Keep Learning & Keep Exploring!*

Chat system - code

```
# ------------------------
# Message Class
# ------------------------
class Message:
    message_counter = 1  # simple counter

    def __init__(self, sender, content):
        self.sender = sender
        self.content = content
        self.id = Message.message_counter
        Message.message_counter += 1

    def __str__(self):
        return f"({self.id}) {self.sender.username}: {self.content}"


# ------------------------
# User Class
# ------------------------
class User:
    def __init__(self, username):
        self.username = username
        self.chatroom = None

    def join_chatroom(self, chatroom):
        if self.chatroom:
            print(f"{self.username} is already in a chatroom.")
        else:
            chatroom.add_user(self)
            self.chatroom = chatroom
            print(f"{self.username} joined {chatroom.name}")

    def leave_chatroom(self):
        if not self.chatroom:
```

```
            print(f"{self.username} is not in any chatroom.")
        else:
            self.chatroom.remove_user(self)
            print(f"{self.username} left {self.chatroom.name}")
            self.chatroom > None

    def send_message(self, content):
        if not self.chatroom:
            print(f"{self.username} cannot send a message (not in a chatroo
m).")
        else:
            self.chatroom.broadcast(self, content)


# ------------------------
# ChatRoom Class
# ------------------------
class ChatRoom:
    def _init_(self, name):
        self.name = name
        self.users = []
        self.messages = []

    def add_user(self, user):
        self.users.append(user)

    def remove_user(self, user):
        self.users.remove(user)

    def broadcast(self, sender, content):
        message > Message(sender, content)
        self.messages.append(message)
        print(message)    # Show message to all users

    def show_chat_history(self):
        print(f"\nChat History of {self.name}:")
        for msg in self.messages:
            print(msg)
```

```
        print()


# --------------------------------
# Example Usage
# --------------------------------
if __name__ == "__main__":
    room = ChatRoom("Python Lounge")

    u1 = User("Alice")
    u2 = User("Bob")
    u3 = User("Charlie")

    u1.join_chatroom(room)
    u2.join_chatroom(room)

    u1.send_message("Hello everyone!")
    u2.send_message("Hi Alice!")

    u3.join_chatroom(room)
    u3.send_message("Hey guys, what's up?")

    room.show_chat_history()

    u1.leave_chatroom()
    u2.leave_chatroom()
    u3.leave_chatroom()
```

Python Fundamentals (Part5)

Concepts : File I/O, Exception Handling, List Comprehensions, JSON module

File I/O

File I/O (Input/Output) means **reading data from files** and **writing data to files** using Python.
Python provides built-in functions to work with files.

Opening a File

We use the `open()` function:

```
file = open("filename", "mode")
```

Example

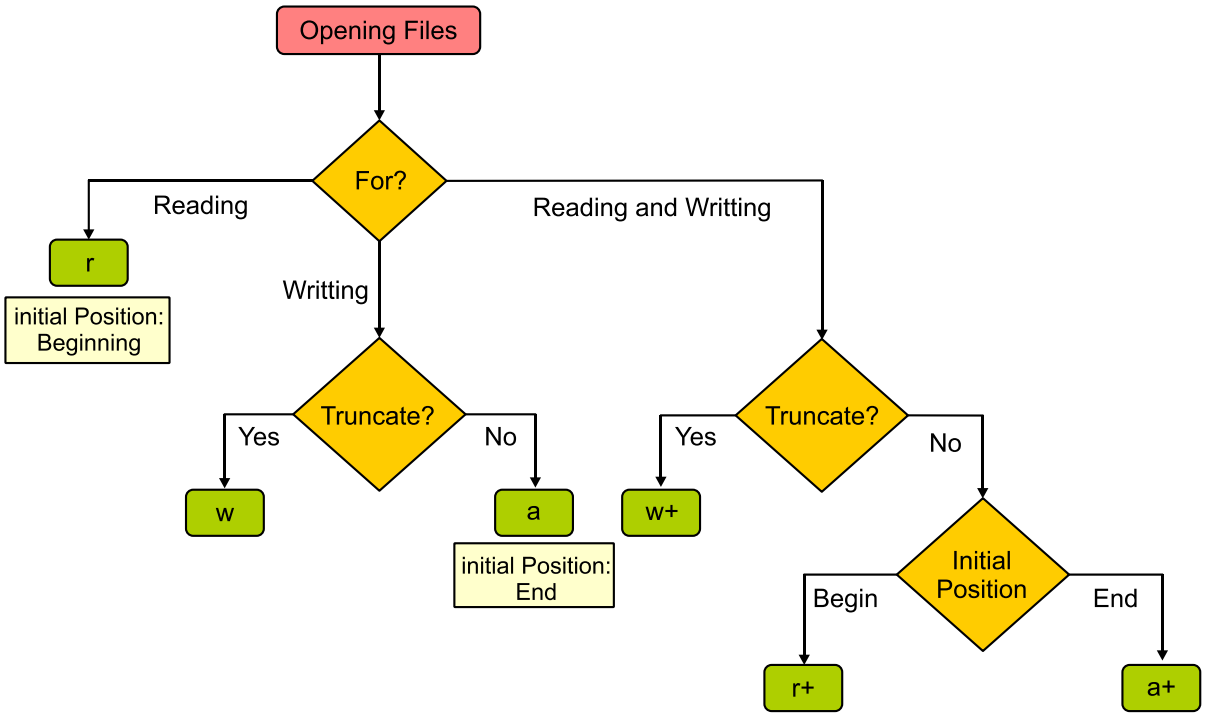
```
f = open("data.txt", "r")
```

Common Modes

Mode	Meaning	Description
`r`	Read	Opens file for reading (default). Error if file doesn't exist.
`w`	Write	Creates new file or overwrites existing file.
`a`	Append	Adds content at the end of the file.
`r+`	Read + Write	File must exist.
`w+`	Write + Read	Overwrites existing file.
`a+`	Append + Read	Reads + adds to end of file.
`b`	Binary Mode	For images, videos, etc. (use with other modes like `rb`).

What to use when?

Follow this flowchart:



Reading from a File

We have multiple functions to read content from a file.

1. `read()`

   Reads entire file as a single string.

```
f = open("data.txt", "r")
content = f.read()
print(content)
f.close()
```

2. `readline()`

   Reads one line at a time.

```
f = open("data.txt", "r")
line1 = f.readline()
line2 = f.readline()
f. close()
```

3. `readlines()`

Reads all lines into a list.

```
f = open("data.txt", "r")
lines = f.readlines()
```

Writing to a File

1. `write()`

Writes a string to a file.

```
f = open("data.txt", "w")
f.write("Hello students!")
f.close()
```

2. `writelines()`

Writes multiple lines at once.

```
f = open("data.txt", "a")
f.writelines(["Line 1\n", "Line 2\n"])
f.close()
```

Always remember to close a file at the end to free system resources.

Recommended: Usage `with open()`

We use a context manager that automatically closes the file.

```
with open("data.txt", "r") as f:
    content = f.read()
    print(content)
```

Deleting a File

To delete a file we use the `remove` function.

```
import os

os.remove("data.txt")
```

Exception Handling

An **exception** is an error that occurs while a program is running. If not handled, the program crashes. Exception Handling allows you to manage errors gracefully so your program continues to run.

An exception occurs when Python encounters something it cannot handle during execution.

Examples:

- Dividing by zero - `ZeroDivisionError`
- Using an undefined variable - `NameError`
- Opening a missing file - `FileNotFoundError`
- Wrong data type - `TypeError`

Basic Syntax

```
try:
    # Code that may cause an error
except:
    # Code that runs if error occurs
```

Example:

```
try:
    x = 10 / 0
except:
    print("Error occurred!")
```

We can also throw specific exceptions:

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("You can't divide by zero!")
```

The `else` Block

`else` executes only if no exception happens.

```
try:
    x = int(input("Enter a number: "))
except ValueError:
    print("Invalid input!")
else:
    print("You entered:", x)
```

The `finally` Block

`finally` always executes, whether an exception occurs or not. It is used for cleanup tasks (closing files, releasing resources).

```
try:
    f = open("data.txt")
    print(f.read())
except FileNotFoundError:
    print ("File not found!")
finally:
    print("Execution completed.")
```

List Comprehensions

List Comprehension is a **short and elegant way** to create lists in Python.
It replaces long `for` loops with **one-line expressions**.

Basic Syntax

```
[expression for item in iterable]
```

Example: Create a list of numbers 1 to 5

```
nums = [x for x in range(1, 6)]
print(nums)    # [1, 2, 3, 4, 5]
```

List Comprehension with Condition

```
[expression for item in iterable if condition]
```

Example: Even numbers from 1 to 10

```
evens = [x for x in range(1, 11) if x % 2 == 0]
```

List Comprehension with if-else

```
[expression_if_true if condition else expression_if_false for item in iterable]
```

Example: Label numbers as “Even” or “Odd”

```
labels = ["Even" if x % 2 == 0 else "Odd" for x in range(1, 6)]
```

Working with JSON Module

JSON (**J**ava**S**cript **O**bject **N**otation) is a lightweight data format used to exchange data between programs, APIs, websites, etc. JSON format is very similar to Python dictionaries.

Python’s `json` module allows you to read, write, encode, and decode JSON.

Importing the Module

```
import json
```

Converting Python to JSON

We use `dumps()` to convert Python objects into JSON string.

```
data = {
    "name": "John",
    "age": 25,
    "marks": [85, 90, 92]
}

json_string = json.dumps(data)
print(json_string)
```

Converting JSON to Python

We use `loads()` to convert JSON string to Python dictionary.

```
json_data = '{"name": "John", "age": 25}'
python_obj = json.loads(json_data)
print(python_obj["name"])
```

Reading JSON from a File (`json.load`)

```
import json

with open("data.json", "r") as f:
    data = json.load(f)

print(data)
```

Writing JSON to a File (`json.dump`)

```
import json

data = {"name": "Aisha", "city": "Delhi"}

with open("data.json", "w") as f:
    json.dump(data, f, indent=4)
```

`indent=4` makes the JSON looks neat and readable.

> *Keep Learning & Keep Exploring!*